Exp: 13 Implementation of Our Own Ping Program

Aim:

To implement a ping program using raw ICMP sockets that sends echo requests and displays the replies.

Requirements:

* Python 3
* Administrator / root privileges (raw sockets)

Algorithm:

1. Create a raw socket with protocol ICMP.
2. Build an ICMP Echo Request (type 8, code 0) with identifier, sequence number and payload, and compute the Internet checksum.
3. Send the packet to the ~~destination~~ destination and record the ti
4. Wait for an Echo Reply (type 0) with a matching identifier and sequence number; compute the round-trip time.
5. Repeat for 4 packets and print the statistics (sent, received, loss, min/avg/max).

Program (run: sudo python3 myping.py google.com):

```
# Simple ping using raw ICMP sockets (run with sudo / as administrator)
import socket, struct, time, sys, os
def checksum(data):
    if len(data) % 2: data += b"\x00"
    s = sum(struct.unpack("!%dH" % (len(data)//2), data))
    s = (s >> 16) + (s & 0xffff)
    s += s >> 16
    return ~s & 0xffff
def ping_once(sock, dest, seq, ident):
    header = struct.pack("!BBHHH", 8, 0, 0, ident, seq)        # type 8 = echo request
    payload = struct.pack("!d", time.time()) + b"abcdefghijklmnop"
```

```
    chk = checksum(header + payload)
    packet = struct.pack("!BBHHH", 8, 0, chk, ident, seq) + payload
    sock.sendto(packet, (dest, 0))
    start = time.time()
    while True:
        sock.settimeout(2)
        try:
            data, addr = sock.recvfrom(1024)
        except socket.timeout:
            return None
        icmp = data[20:28]                                  # skip 20-byte IP header
        t, code, _, pid, pseq = struct.unpack("!BBHHH", icmp)
        if t == 0 and pid == ident and pseq == seq:         # type 0 = echo reply
            return (time.time() - start) * 1000, data[8], len(data) - 20
host = sys.argv[1] if len(sys.argv) > 1 else "127.0.0.1"
dest = socket.gethostbyname(host)
sock = socket.socket(socket.AF_INET, socket.SOCK_RAW, socket.IPPROTO_ICMP)
ident = os.getpid() & 0xffff
sent = recv = 0; times = []
print(f"PING {host} ({dest})")
try:
    for seq in range(1, 5):
        sent += 1
        r = ping_once(sock, dest, seq, ident)
        if r is None:
            print("Request timed out")
        else:
            recv += 1; times.append(r[0])
            print(f"Reply from {dest}: bytes={r[2]} ttl={r[1]} "
                  f"time={r[0]:.2f} ms")
        time.sleep(1)
finally:
    print(f"\n--- {host} ping statistics ---")
    loss = 100 * (sent - recv) // sent
    print(f"{sent} packets transmitted, {recv} received, {loss}% packet loss")
    if times:
        avg = sum(times) / len(times)
        print(f"min/avg/max = {min(times):.2f}/{avg:.2f}/{max(times):.2f} ms")
```

Result:

The ping program was implemented using raw ICMP sockets and tested successfully.